Lesson: Basics of Neural Networks

Imagine you're trying to recognize handwritten digits or predict tomorrow's weather. These are complex problems for traditional models to solve effectively. That's where neural networks shine. Inspired by the structure of the human brain, neural networks are powerful machine learning models capable of identifying patterns in data that are too intricate for other algorithms.

You've probably heard terms like layers, neurons, and activation functions—but what do they mean? And how do they come together to create models that can recognize images, translate languages, or even write code?

In this lesson, we'll break it down step by step, demystifying how neural networks work from the inside out.

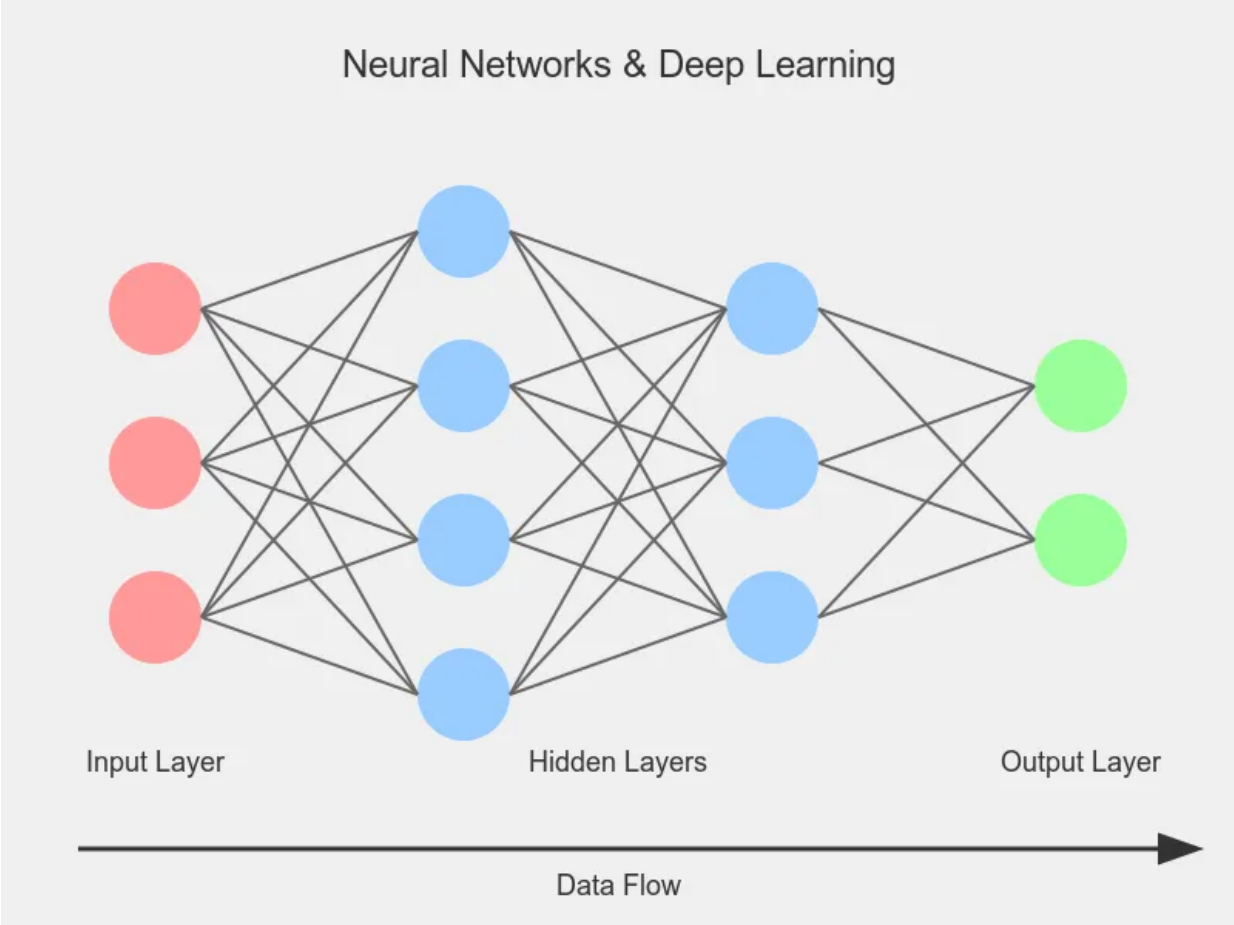
Learning Outcomes

By the end of this lesson, you will be able to:

- Understanding how neural networks work (layers, activation functions, backpropagation).
- Training a simple feed-forward neural network using Keras.

Introduction to Neural Networks

A neural network is an artificial intelligence (AI) technique that enables computers to analyze data like how the human brain operates. It falls under a category of machine learning (ML) known as deep learning and consists of layered networks of interconnected nodes, or neurons, that mimic brain structure. This setup allows the system to adapt by learning from errors and refining its performance over time. As a result, artificial neural networks are used to tackle complex tasks such as document summarization or facial recognition with high precision. Neural networks enable computers to make smart decisions with minimal human intervention. This capability comes from their ability to learn and represent complex, nonlinear relationships between inputs and outputs.



Use Cases and Applications of Neural Networks

Neural networks are widely used across industries due to their exceptional ability to learn complex, non-linear patterns from data and make high-quality predictions. Their versatility allows them to adapt to a range of tasks, from classification and regression to signal interpretation and control. These models are particularly valuable in domains that involve high-dimensional data, uncertainty, or pattern recognition, making them foundational in modern AI systems.

Industry Applications

Healthcare

- **Medical Image Classification**: Neural networks power diagnostic systems that can detect tumors, fractures, and organ anomalies from X-rays, MRIs, and CT scans.
- **Predictive Analytics**: Used in forecasting disease progression and patient outcomes.

Finance

- **Market Prediction**: Time series forecasting for stock prices, risk modeling, and fraud detection.
- **Algorithmic Trading**: Neural networks drive strategies by learning complex market behaviors.

Marketing

- **Customer Segmentation**: Analyze behavioral and demographic data to personalize campaigns.
- **Recommendation Systems**: Power product recommendations based on user preferences.

Energy

- **Load Forecasting**: Predict energy demand to optimize generation and reduce costs.
- **Anomaly Detection**: Identify faults in grids and industrial sensors using time-series data.

Manufacturing

- **Quality Control:** Use computer vision to detect defects in products during production.
- **Chemical Composition Analysis**: Classify materials and mixtures through sensor data or spectroscopy.

Key Technological Domains

1. Computer Vision

Neural networks enable machines to understand and process visual data, powering:

- **Autonomous Vehicles**: Object detection, lane tracking, and obstacle avoidance.
- **Facial Recognition**: Identity verification and access control.
- **Content Moderation**: Automatic filtering of inappropriate images and videos.

2. Speech Recognition

Neural architectures such as RNNs, LSTMs, and Transformers help machines understand human speech, enabling:

- **Virtual Assistants**: Voice-controlled interaction in smart devices.
- **Real-Time Transcription**: Converting spoken language into text (e.g., for subtitles or note-taking).
- **Call Classification and Routing**: Automating support systems in customer service.

Understanding Neural Networks Architecture

At the heart of every neural network lies a structured architecture composed of **layers**. These layers are responsible for transforming inputs into outputs through a series of learned computations. Understanding how these layers interact is essential for grasping how neural networks learn patterns, generalize knowledge, and perform predictions.

Layers in a Neural Network

A **layer** in a neural network is a group of **neurons (or nodes)** that process and transmit information. Each neuron receives input, applies a mathematical transformation (typically involving

a weight, bias, and activation function), and passes its result to the next layer. The depth and type of layers define the network's capacity to learn complex representations.

Input Layer

- The first layer of the network.
- Receives raw data (e.g., pixel values in an image, word embeddings, or tabular features).
- Does **not perform learning**, only passes inputs to the next layer.

Hidden Layers

- One or more layers between the input and output layers.
- Each neuron in a hidden layer:
    - Applies a **weighted sum** of inputs.
    - Adds a **bias** term.
    - Passes the result through a **non-linear activation function** (e.g., ReLU, sigmoid).
- **More hidden layers** or **more neurons per layer** increase the model's representational power.
- Deep neural networks (DNNs) typically have **multiple hidden layers**.

Output Layer

- The final layer that produces the model's **prediction**.
- The number of output neurons and their activation function depend on the task:
    - **Classification**: Softmax for multi-class, sigmoid for binary.
    - **Regression**: Often a single neuron with no activation or linear activation.

Neurons and Weights

Each node in a neural network is called a **neuron**. Neurons are connected by **weights**, which control the strength of the connection. The output of a neuron is calculated by multiplying the input by the weight and applying an activation function.

Activation Functions

Activation functions determine whether a neuron should be activated or not. They introduce non-linearity to the model, allowing it to learn complex patterns. Non-linearity means the model can learn curved or complex patterns, not just straight lines, like fitting a winding path instead of a rigid ruler. Activation functions decide whether a neuron's output should be passed on, adding flexibility to learn complex patterns. **Non-linearity** means the model can learn curved or complex patterns, not just straight lines, like fitting a winding path instead of a rigid ruler.

- **Sigmoid**: Squashes outputs to a range between 0 and 1, like a dimmer switch that adjusts brightness. It's great for binary classification (e.g., "yes" or "no") because it gives probabilities, but it can be slow to train.

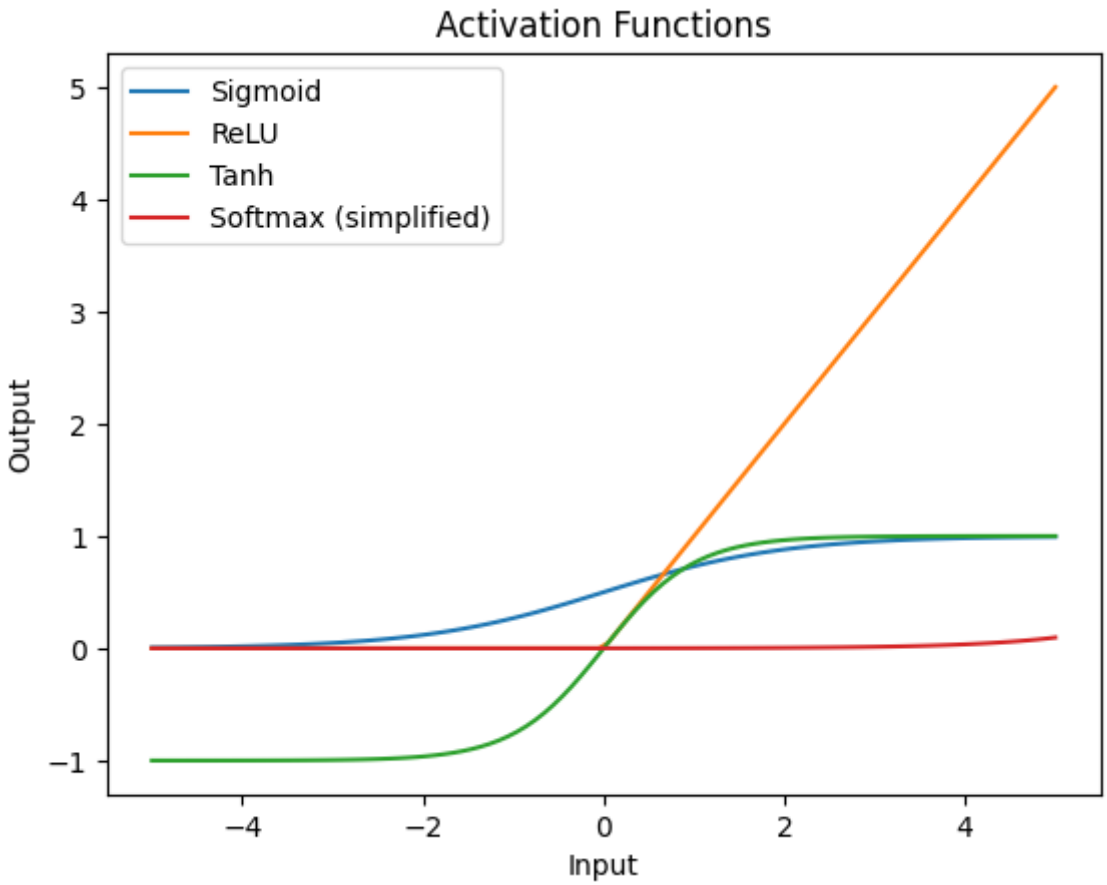
- **ReLU (Rectified Linear Unit)**: Acts like a switch: outputs 0 for negative inputs and keeps positive inputs unchanged. It's fast and effective in hidden layers, helping models learn quickly without getting stuck.
- **Tanh**: Similar to Sigmoid but outputs between -1 and 1, like a balance scale that swings both ways. It's useful when inputs can be positive or negative, offering a broader range than Sigmoid.
- **Softmax**: Used in multi-class classification (e.g., identifying Iris flower types). It turns outputs into probabilities that add to 1, like dividing a pie into slices where each slice is a class's likelihood.

Here's a plot to visualize these functions:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
sigmoid = 1 / (1 + np.exp(-x))
relu = np.maximum(0, x)
tanh = np.tanh(x)
softmax = np.exp(x) / np.sum(np.exp(x))  # Simplified for visualization

plt.plot(x, sigmoid, label='Sigmoid')
plt.plot(x, relu, label='ReLU')
plt.plot(x, tanh, label='Tanh')
plt.plot(x, softmax, label='Softmax (simplified)')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('Activation Functions')
plt.legend()
```



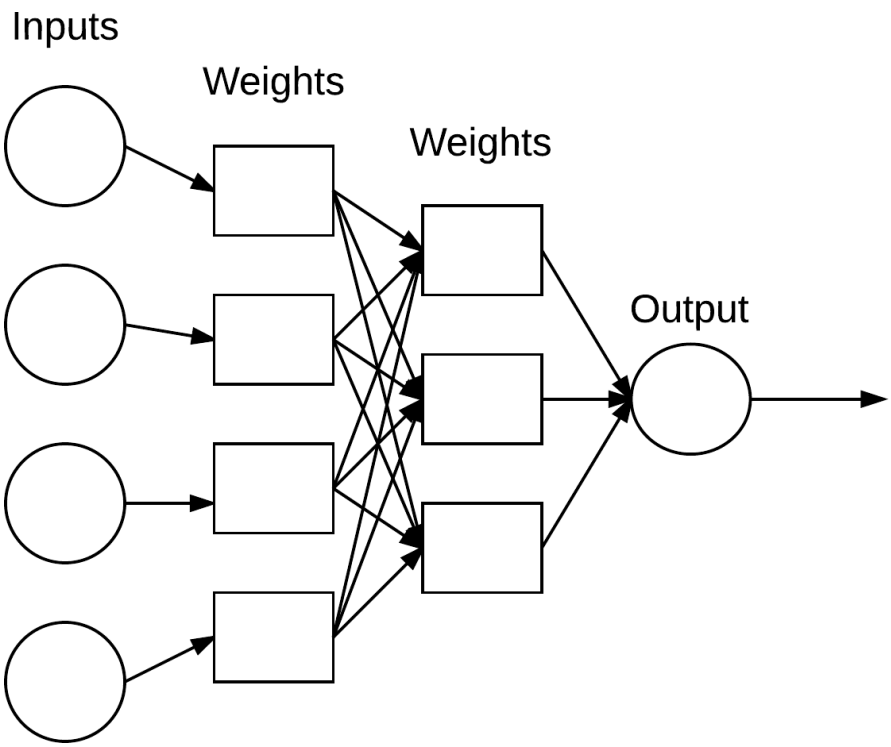
Output

Process

At their core, neural networks are a series of mathematical operations organized into layers. Data flows through these layers, gets transformed at each step, and ultimately results in a prediction. This transformation process allows neural networks to learn complex patterns from data.

Step by Step Workflow:

- **Input Data**: The network receives raw input features (e.g., pixel values of an image, numerical data, or words represented as vectors).
- **Weighted Sum**: Each neuron computes a weighted sum of its inputs.
- **Activation**: This sum is passed through a non-linear activation function to introduce complexity and flexibility.
- **Layer by Layer Propagation**: Outputs from one layer become inputs to the next layer.
- **Output Prediction**: The final layer generates a prediction, such as a class label or numeric value.
- **Learning through Feedback**: During training, the prediction is compared to the actual result, and the network updates its internal weights using a process called backpropagation.



Calculating Output in a Neural Network

To understand how a neural network generates predictions, we need to examine the basic mathematical operations inside each neuron.

For a Single Neuron

A neuron works like a chef mixing ingredients: it takes inputs (like flour or sugar), multiplies each by a weight (how much of each ingredient to use), adds a bias (like a pinch of salt), and then applies an activation function (like baking the mix) to produce an output.

Component	Symbol
Inputs	$x_1, x_2, \ldots, x_n$
Weights	$w_1, w_2, \ldots, w_n$
Bias	b
Activation Function	f

The neuron's output is calculated as:

$$z = \sum_{i=1}^{n} w_i x_i + ba = f(z)$$

Where:

- z is the **linear combination** (pre-activation)
- a is the **output** after applying the activation function

Example: Layer Computation Using ReLU Activation

Activation functions introduce non-linearity into a neural network, allowing it to model complex patterns in data. One of the most commonly used activation functions is the **Rectified Linear Unit (ReLU)**.

The ReLU function is defined as:

$$f(z) = max(0, z)$$

This means that ReLU outputs the input value z if it is positive, and zero otherwise. It is computationally efficient and helps mitigate the vanishing gradient problem in deep networks.

Forward Propagation Through a Layer (Matrix Form)

When computing the output of an entire layer in a neural network, matrix operations are used for efficiency and scalability. When computing for a full layer of neurons using matrix notation:

$$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$$

Where:

Symbol	Description
$a^{(l-1)}$	Output from the previous layer
$W^{(l)}$	Weight matrix for the current layer
$b^{(l)}$	Bias vector for the current layer
f	Activation function
$a^{(l)}$	Output of the current layer

This operation is repeated sequentially through each layer, from the input layer all the way to the output layer.

Types of Neural Networks

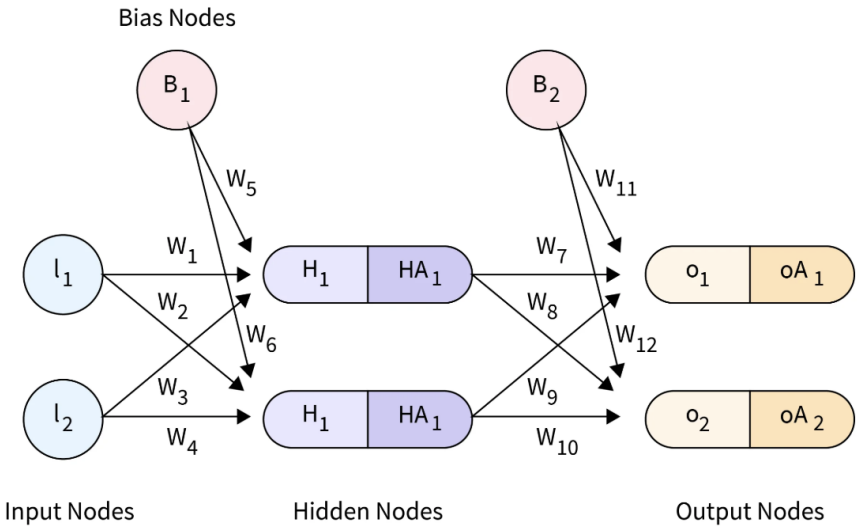
Neural networks come in a variety of architectures, each tailored to specific data types and learning tasks. The most foundational structure is the **Feedforward Neural Network (FNN)**, which has inspired more advanced architectures like **Convolutional Neural Networks (CNNs)** for spatial data and **Recurrent Neural Networks (RNNs)** for temporal sequences.

Feedforward Neural Networks (FNN):

Feedforward neural networks handle data by passing it in a single direction from input to output without looping back. Each neuron in one layer connects to every neuron in the subsequent layer. Although the data flows forward, these networks refine their predictions over time through a feedback mechanism during training.

Process

- Each neuron receives input, processes it, and passes its output to the next layer.
- The network learns by comparing the output to the actual label and adjusting the weights during training (using backpropagation).
- FNNs are suitable for simple classification and regression problems where the input data has no inherent sequence.

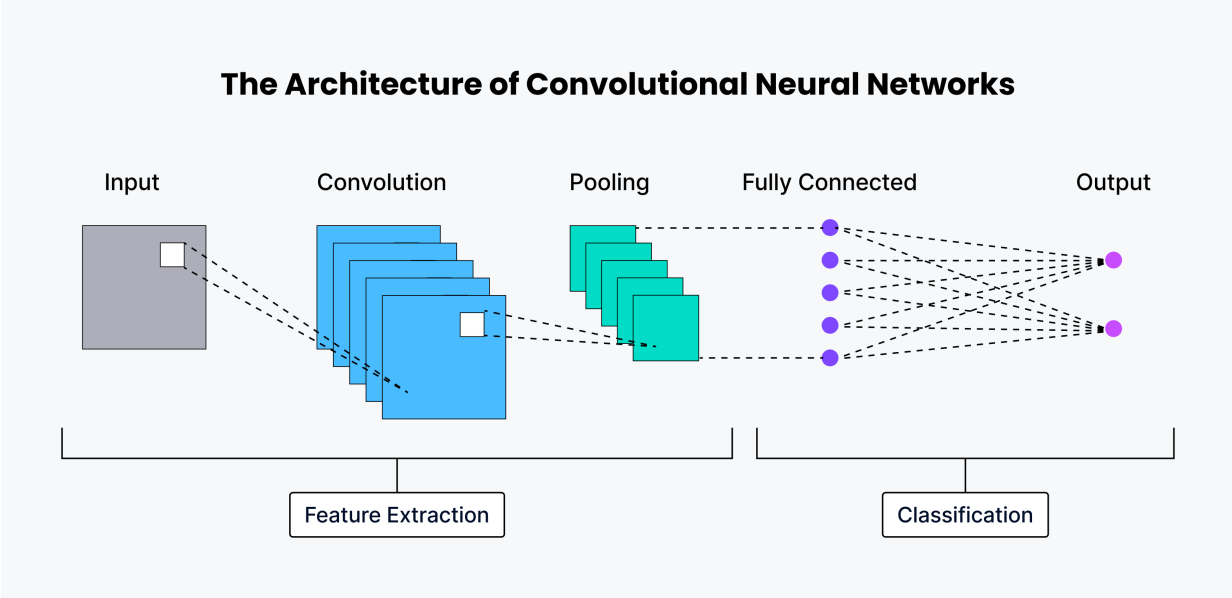


Convolutional Neural Networks (CNNs):

The hidden layers in convolutional neural networks (CNNs) carry out specialized mathematical operations known as convolutions, such as filtering or summarizing data. These layers are especially effective in image classification tasks because they can identify and extract key features from images that aid in accurate recognition and categorization. By transforming the image into a more manageable form while preserving essential details, CNNs make processing more efficient. Each hidden layer focuses on capturing different aspects of the image, such as edges, colors, and depth.

Process

- Input images are passed through **convolutional layers** that act like filters to extract important features.
- These features are passed through **pooling layers** to reduce spatial dimensions while keeping the most essential information.
- Finally, fully connected layers interpret these features and make predictions.
- CNNs are highly effective in **image classification, object detection, and facial recognition** due to their ability to recognize spatial hierarchies.

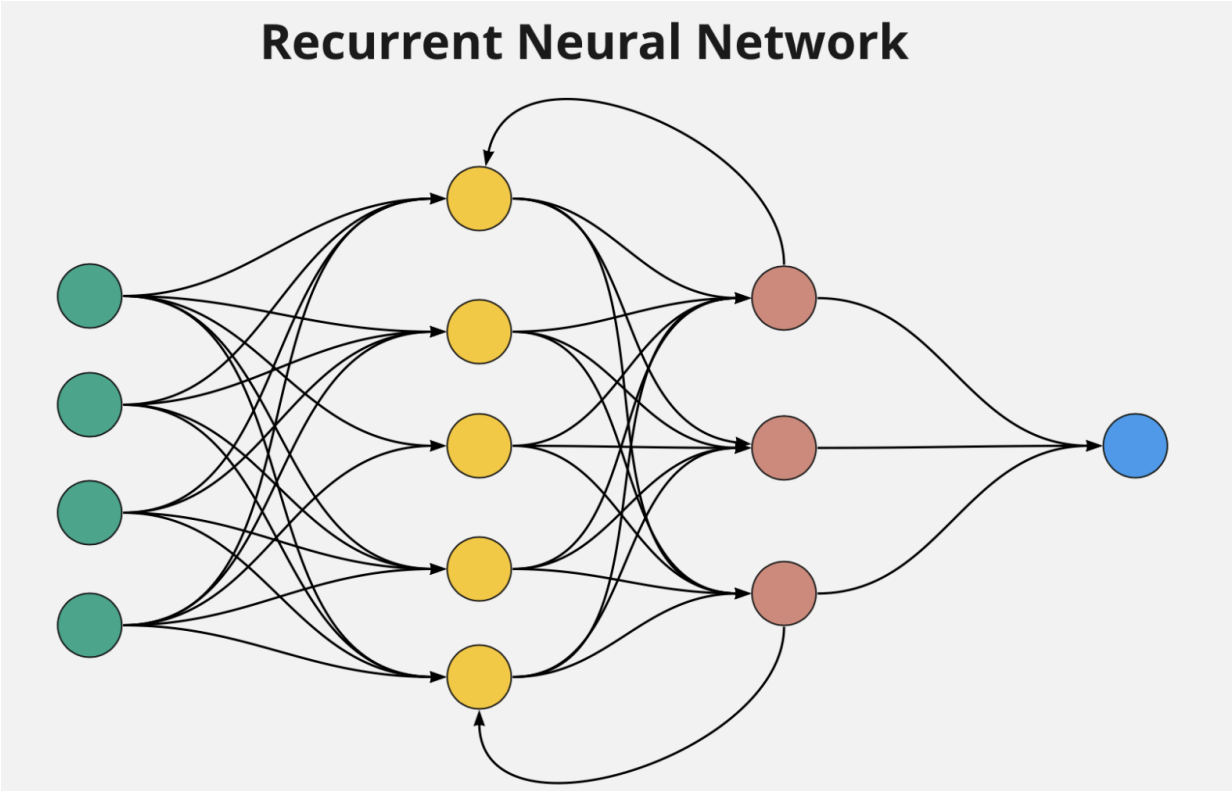


Recurrent Neural Networks (RNNs):

Recurrent Neural Networks (RNNs) are specially designed to handle sequential data, where the order of data points matters. This makes them ideal for tasks like time series forecasting, speech recognition, and language modeling.

Process

- Each input in a sequence is processed one at a time.
- The network maintains a **hidden state** that carries context from previous inputs.
- This memory allows RNNs to make predictions based not just on the current input, but also on what came before it.
- RNNs are widely used in **language modeling, speech recognition, and forecasting**, where understanding past inputs is crucial.



Training a Neural Network

Training a neural network is the process of adjusting its internal weights and biases so it can make accurate predictions. This is achieved through an iterative cycle of learning steps, repeated over multiple epochs (complete passes through the training data), with the goal of minimizing prediction error. The training process involves four key stages:

1. Forward Propagation

In forward propagation, input data flows layer by layer through the network:

- Each neuron computes a weighted sum of its inputs.
- A bias is added, and the result is passed through an activation function (e.g., ReLU, sigmoid).
- The output of one layer becomes the input to the next.
- This continues until the final output is produced.

Forward propagation transforms raw input into a prediction using the current state of the network's weights.

2. Loss Function Evaluation

After producing an output, the network evaluates how far the prediction is from the **true target** using a **loss function**. This scalar value measures prediction error. Common Loss Functions:

- **Mean Squared Error (MSE)**: Used in **regression** tasks.
- **Cross-Entropy**: Used in **classification** tasks.

The loss function provides a **quantitative signal** the network uses to assess its current performance.

3. Backpropagation

Backpropagation is the process by which the network learns from its mistakes:

- It calculates the gradient of the loss with respect to each weight by working backward from the output layer to the input layer.
- These gradients indicate how much each weight contributed to the error.
- Each weight is then adjusted to reduce the error on the next pass.

Think of backpropagation as retracing the model’s decision-making path to fine-tune the steps that led to a wrong prediction.

4. **Optimization with Gradient Descent**

Once gradients are computed, an **optimization algorithm** updates the weights to reduce the loss:

- **Gradient Descent** uses the computed gradients to move weights **in the direction that reduces error**.
- The **learning rate** determines how large each update step is.

Common Optimizers:

- **Stochastic Gradient Descent (SGD)**: Updates weights using small batches of data for efficiency.
- **Adam**: An adaptive optimizer that adjusts learning rates dynamically and often converges faster.

Optimization is like finding the lowest point in a landscape of error—smarter algorithms and well-tuned learning rates make this search more efficient.

Training Cycle

These four steps are repeated over many **epochs**, with the model gradually improving as its parameters are adjusted. The goal is to find a set of weights that produce **low error** on both training and unseen data (generalization).

Implementing a Simple Feed-forward Neural Network

To solidify your understanding of neural network architecture and training, let's implement a basic **Feedforward Neural Network (FNN)** using the **Keras API**, which is part of the TensorFlow library. This example uses a common dataset and a straightforward architecture to demonstrate key concepts in practice.

Keras Implementation

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical
from keras.layers import Input

# Load dataset
iris = load_iris()
X = iris.data
y = to_categorical(iris.target)  # One-hot encode target labels

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define a simple feedforward neural network
model = Sequential([
    Input(shape=(4,)),                # Input layer: 4 features
    Dense(64, activation='relu'),     # Hidden layer 1: Dense (Fully
Connected), 64 units, ReLU activation
    Dense(32, activation='relu'),     # Hidden layer 2: Dense, 32 units,
ReLU activation
    Dense(3, activation='softmax')    # Output layer: Dense, 3 classes,
Softmax activation
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=
['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=50, batch_size=10)
```

Key Highlights

- **Input Layer**: Takes in 4 numerical features (sepal/petal measurements).
- **Hidden Layers**: Use `ReLU` for non-linear transformation and feature learning.
- **Output Layer**: `Softmax` activation distributes probabilities across the 3 classes.
- **Loss Function**: `categorical_crossentropy`, appropriate for one-hot encoded multi-class targets.
- **Optimizer**: `Adam`, an efficient and commonly used optimizer.

PyTorch Implementation:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
import numpy as np

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)

# One-hot encode the target
encoder = OneHotEncoder(sparse=False)
y_encoded = encoder.fit_transform(y)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert to PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

# Wrap in DataLoader
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=10, shuffle=True)
```

Define the Model

```
class FeedforwardNN(nn.Module):
    def __init__(self):
        super(FeedforwardNN, self).__init__()
        self.fc1 = nn.Linear(4, 64)   # Input layer → 64 neurons
        self.fc2 = nn.Linear(64, 32)  # Hidden layer → 32 neurons
        self.out = nn.Linear(32, 3)   # Output layer → 3 neurons (classes)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.softmax(self.out(x), dim=1)  # Softmax for multi-class
classification
        return x

model = FeedforwardNN()
```

Compile and Train the Model

```
# Define loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# Training loop
epochs = 50
for epoch in range(epochs):
    for X_batch, y_batch in train_loader:
        # Forward pass
        outputs = model(X_batch)
        loss = criterion(outputs, torch.max(y_batch, 1)[1])  # Convert one-
hot to class index

        # Backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")
```

Evaluation

```
with torch.no_grad():
    y_pred = model(X_test_tensor)
    predicted_classes = torch.argmax(y_pred, dim=1)
    actual_classes = torch.argmax(y_test_tensor, dim=1)
    accuracy = (predicted_classes == actual_classes).sum().item() /
len(actual_classes)

print(f"Test accuracy: {accuracy:.4f}")
```

Key Notes

- **Model Architecture**: Mirrors the Keras model—2 hidden layers with ReLU, output layer with Softmax.
- **Loss Function**: `CrossEntropyLoss`, which expects raw scores (logits) and class labels (not one-hot).
- **Optimizer**: `Adam`, with a learning rate of `0.01`.
- **Evaluation**: Predictions are compared after converting probabilities to class labels using `argmax`.

Conclusion

Neural networks are a foundational component of modern artificial intelligence, capable of learning complex patterns and making predictions across a wide range of tasks. In this lesson, you explored how neural networks are structured, how they process information through layers of neurons, and how they learn through forward propagation, backpropagation, and optimization.

You also learned the differences between key neural network architectures, Feedforward, Convolutional, and Recurrent, and how they apply to different types of problems. Finally, you implemented a simple feedforward neural network using Keras, giving you hands-on experience in building and training a neural model.